Day 45 : useRef Hook

What is useRef?

`useRef` is a React hook that lets you store values that:

1. **Persist between renders** (don't get reset)
2. **Don't trigger re-renders** when changed
3. Are **mutable** (you can change them directly)

Syntax

```
const myRef = useRef(initialValue);
```

Returns an object: `{ current: initialValue }`

Two Main Use Cases

1. Storing Mutable Values

Store values that need to persist but shouldn't cause re-renders.

Example: Stopwatch

```
function Stopwatch() {
  const [time, setTime] = useState(0);
  const intervalRef = useRef(null);

  const start = () => {
    if (intervalRef.current) clearInterval(intervalRef.current);

    intervalRef.current = setInterval(() => {
      setTime(prev => prev + 1);
    }, 1000);
  };
```

```
  const stop = () => {
    clearInterval(intervalRef.current);
    intervalRef.current = null;
  };

  const reset = () => {
    clearInterval(intervalRef.current);
    intervalRef.current = null;
    setTime(0);
  };

  return (
    <div>
      <p>Time: {time}s</p>
      <button onClick={start}>Start</button>
      <button onClick={stop}>Stop</button>
      <button onClick={reset}>Reset</button>
    </div>
  );
}
```

2. Accessing DOM Elements

Get direct reference to DOM elements to call their methods.

Example: Auto-focus Input

```
function SearchBox() {
  const inputRef = useRef(null);

  const focusInput = () => {
    inputRef.current.focus();
  };

  return (
    <div>
```

```
      <input ref={inputRef} type="text" placeholder="Search..." />
      <button onClick={focusInput}>Focus</button>
    </div>
  );
}
```

Example: Video Player

```
function VideoPlayer({ src }) {
  const videoRef = useRef(null);

  return (
    <div>
      <video ref={videoRef} src={src} width="600" />
      <button onClick={() => videoRef.current.play()}>Play</button>
      <button onClick={() => videoRef.current.pause()}>Pause</button>
      <button onClick={() => videoRef.current.currentTime = 0}>Restart</button
>
    </div>
  );
}
```

Example: Form Handling

```
function LoginForm() {
  const emailRef = useRef(null);
  const passwordRef = useRef(null);

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Email:', emailRef.current.value);
    console.log('Password:', passwordRef.current.value);
  };

  return (
    <form onSubmit={handleSubmit}>
```

```
      <input ref={emailRef} type="email" placeholder="Email" />
      <input ref={passwordRef} type="password" placeholder="Password" />
      <button type="submit">Login</button>
    </form>
  );
}
```

useRef vs useState

Feature	useRef	useState
Triggers re-render	❌ No	✅ Yes
Persists between renders	✅ Yes	✅ Yes
Mutable	✅ Yes (change .current directly)	❌ No (use setter)
Use case	Store values, DOM access	UI state

Key Points to Remember

1. Always access/modify via `.current` property
2. Changing `ref.current` does NOT trigger re-render
3. `ref={myRef}` on JSX elements makes React auto-assign the DOM element to `myRef.current`
4. Perfect for: timers, DOM manipulation, storing previous values, uncontrolled forms
5. When you need value changes to update UI → use `useState`
6. When you need value to persist but not affect UI → use `useRef`

Common DOM Methods with useRef

```
// Input/Textarea
inputRef.current.focus()
inputRef.current.blur()
inputRef.current.select()
```

```
inputRef.current.value

// Video/Audio
videoRef.current.play()
videoRef.current.pause()
videoRef.current.currentTime = 30
videoRef.current.volume = 0.5

// Any Element
elementRef.current.scrollIntoView()
elementRef.current.getBoundingClientRect()
elementRef.current.classList.add('active')
```

Practice Exercise: Build a timer that can start, stop, reset, and skip forward/backward by 5 seconds. Use useRef for the interval ID.